StyleMate – AI Fashion Outfit Recommender

Project File

OF

Introduction of AI-ML

(UCS1002)

(2025-2029)

Submitted by:
Name: Deepak Kumar
Roll No: 25SCS1003002716
Course / Branch: B.Tech. (CSE)

Submitted to:

Name of Guide / Faculty: Mr. Anand Mishra

Department of Computer Science & Engineering

IILM University Greater Noida

CERTIFICATE

This is to certify that the project report titled **"StyleMate- AI Fashion Outfit Recommender"** submitted by **Deepak Kumar,** bearing Roll No. **25SCS1003002716,** in partial Fulfilment of the requirements for the internship/project evaluation, is a Bonafide work carried out under my supervision.

Project Guide
Name- Mr. Anand Mishra
Designation- Associate Professor
Department of Computer Science & Engineering

Head of Department
Name-
Designation-

Date:
Place:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my guide **Mr. Anand Mishra** for the continuous support and guidance provided to me throughout this project. I also thank the faculty members, my institution, and my peers for their encouragement and motivation.

INDEX (TABLE OF CONTENTS)

1. INTRODUCTION

In today's digital world, selecting a stylish outfit can be challenging due to the wide variety of clothing styles, colours, and patterns. An **StyleMate - AI Fashion Outfit Recommender** helps users by automatically suggesting clothing combinations that are visually appealing and compatible.

The system uses **deep learning models**, such as **ResNet50**, to extract key features from outfit images, including colour, texture, and style. Similarity metrics like **cosine similarity** are applied to recommend matching items from a large dataset.

This AI-driven approach provides a **personalized fashion experience**, saves time, and introduces users to new outfit combinations. It is widely used in e-commerce platforms, virtual stylists, and mobile apps to enhance customer engagement and simplify shopping.

Overall, the project demonstrates how **AI and machine learning** can transform fashion recommendation systems by offering smart, efficient, and personalized outfit suggestions.

2. PROBLEM STATEMENT

Selecting a well-coordinated outfit is often challenging due to the variety of clothing styles, colours, and patterns. Manual selection or reliance on human stylists is time-consuming and subjective, making it difficult to get consistent, personalized fashion advice.

The problem this project addresses is developing an **AI-based system** that can automatically analyse clothing images, extract key visual features, and recommend compatible outfit combinations. By leveraging **deep learning and similarity metrics**, the system aims to provide accurate, personalized recommendations that save time and improve user satisfaction.

Such a solution not only helps users make better fashion choices but also benefits e-commerce platforms by enhancing customer engagement and shopping experience.

3. OBJECTIVE

The objective of this project is to build an AI-powered fashion recommendation system that analyses clothing images and suggests visually similar outfits. By combining deep learning (ResNet50) for feature extraction with cosine similarity for matching, the system delivers accurate and personalized recommendations. Specifically, the project aims to:

• Apply deep learning to extract meaningful visual features from fashion images.

• Implement similarity computation to identify and rank matching outfits

. • Provide a simple, interactive interface using Gradio for user engagement.

• Visualize similarity scores with Matplotlib to ensure transparency in recommendations.

• Integrate multiple libraries (TensorFlow/Keras, Pillow, NumPy, Scikit-learn, Matplotlib, Gradio) into a cohesive workflow.

Beyond technical implementation, the broader aim is to demonstrate how AI can enhance fashion discovery and personalization, offering users a smarter way to explore styles. This project also lays the foundation for future improvements such as metadata filtering, real-time deployment, and integration with e-commerce platforms.

4. SYSTEM ARCHITECTURE

1. Dataset Collection

- Collect labelled outfit images (JPG/PNG) with metadata (type, colour, style).
- Store in a structured folder (e.g., `sample_data/`).

2. Preprocessing

- Resize images to 224×224 pixels.
- Normalize pixel values and remove background noise.
- Augment data to improve model robustness.

3. Feature Extraction

- Use **ResNet50** (pretrained on ImageNet) without top layers.
- Convert each image into a fixed-length feature vector.
- Store extracted features for similarity comparison.

4. Similarity Computation

- When a user uploads an image, extract its features.
- Compute **cosine similarity** between uploaded image and dataset.
- Rank and retrieve top 5 visually similar outfits.

5. Visualization

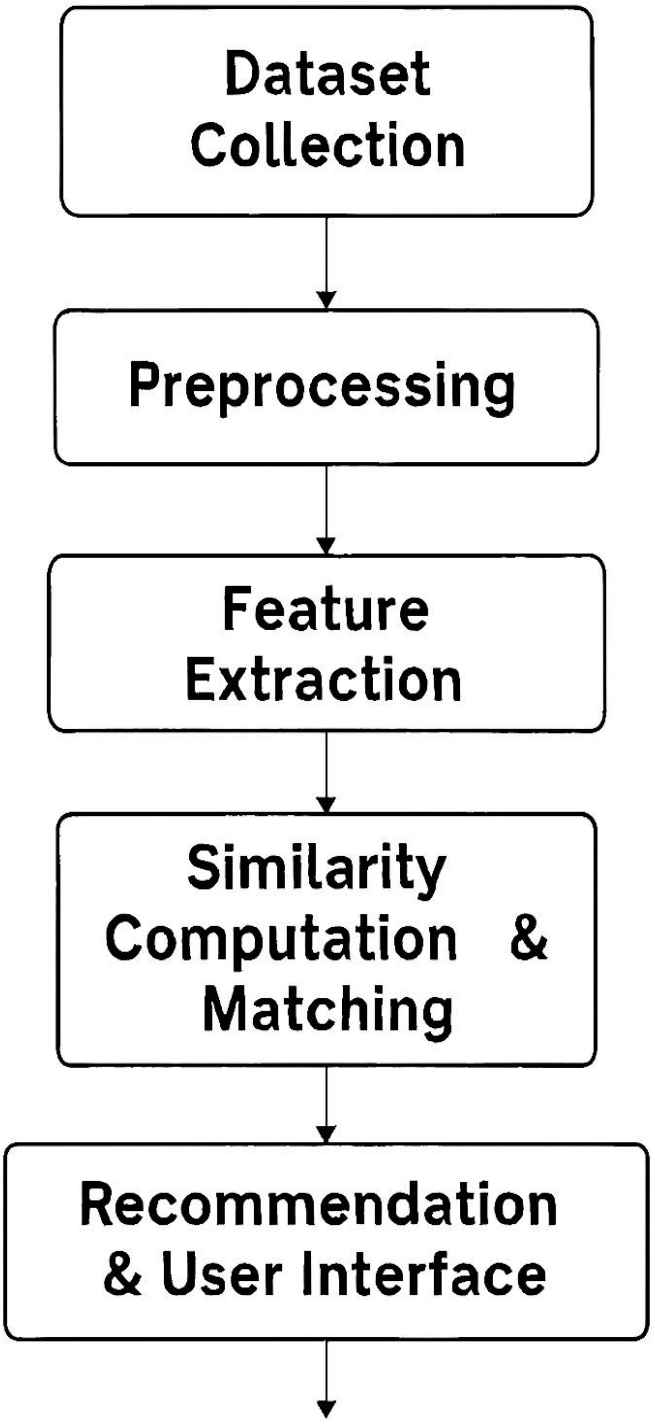
- Generate a **bar chart** using Matplotlib to show similarity scores.

- Display filenames and match strength for user clarity.

6. Recommendation & User Interface

- Use **Gradio** to build an interactive web interface:
  - Upload image
  - View top matching outfits
  - See similarity chart
  - Enable real-time feedback and fashion discovery.

SYSTEM ARCHITECHURE



5. METHODOLOGY

1. Dataset Collection

- Gather high-quality outfit images (e.g., boys' clothing) in JPG format.
- Label images with metadata such as type, colour, and style to support feature-based matching.

2. Image Preprocessing

- Resize all images to a uniform size (224×224 pixels).
- Normalize pixel values and apply augmentation techniques to improve model generalization.
- Remove noisy backgrounds to focus on clothing features.

3. Feature Extraction

- Use **ResNet50**, a deep convolutional neural network pretrained on ImageNet, with top layers removed and global average pooling.
- Convert each image into a feature vector representing visual attributes like colour , texture, and shape.

4. Similarity Computation

- Apply **cosine similarity** to compare the uploaded image's feature vector with those in the dataset.
- Identify top 5 visually similar outfits based on similarity scores.

5. Visualization

- Generate a horizontal bar chart using **Matplotlib** to display similarity scores for the top matches.
- Provide visual feedback to users for better interpretability.

6. User Interface

- Build an interactive UI using **Gradio**:
    - Users upload an outfit image.
    - System displays top matching outfits and similarity chart.
    - Enables quick and informed fashion decisions.

6. TECHNOLOGIES USED

1. Programming Language

- **Python**: Core language for building the system, handling image processing, machine learning, and UI integration.

2. Deep Learning Framework

- **TensorFlow / Keras**: Used for loading and applying the **ResNet50** model to extract deep features from outfit images.

3. Image Processing

- **Pillow (PIL)**: For handling image uploads, resizing, and format conversion.
- **TensorFlow Keras Preprocessing**: For converting images into arrays and preparing them for model input.

4. Similarity Computation

- **Scikit-learn**: Utilized for computing **cosine similarity** between image feature vectors to find visually similar outfits.

5. Visualization

- **Matplotlib**: Generates bar charts to visualize similarity scores of recommended outfits.

6. User Interface

- **Gradio**: Builds an interactive web-based UI where users can upload images and view recommendations and charts.

7. Model

- **ResNet50 (Pretrained on ImageNet)**: A convolutional neural network used for extracting high-level visual features from clothing images.

8. Environment Setup

- **Virtual Environment + pip**: Ensures isolated and reproducible installation of required libraries.

---

7. IMPLEMENTATION & CODE

```
#Install Libraries
!pip install tensorflow pillow scikit-learn gradio matplotlib
```

Show hidden output

```
# Import Modules
import numpy as np
import os
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input
from tensorflow.keras.preprocessing import image
from sklearn.metrics.pairwise import cosine_similarity
from PIL import Image
```

```
#Folder Setup
import os
print("Images found:", os.listdir("sample_data"))
```

```
Images found: ['README.md', 'anscombe.json', 'mnist_train_small.csv', 'california_housing_train.csv', 'california_housing_te
```

```
import os
os.makedirs("found_outfits", exist_ok=True)
```

```
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input
from tensorflow.keras.preprocessing import image
import numpy as np
import os

# Preprocess Dataset
# Load ResNet model (for feature extraction)
model = ResNet50(weights="imagenet", include_top=False, pooling="avg")

def extract_features(img_path):
    try:
        img = image.load_img(img_path, target_size=(224, 224))
        x = image.img_to_array(img)
        x = np.expand_dims(x, axis=0)
        x = preprocess_input(x)
        features = model.predict(x)
        return features.flatten()
    except Exception as e:
        print(f"Error processing {img_path}: {e}")
        return None

def load_and_process_images(folder):
    image_paths, features = [], []
    for file in os.listdir(folder):
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            path = os.path.join(folder, file)
            feat = extract_features(path)
            if feat is not None:
                image_paths.append(path)
                features.append(feat)
    print(f"✅ Total images processed: {len(image_paths)}")
    return image_paths, np.array(features)

image_paths, features = load_and_process_images("sample_data")
```

Show hidden output

```
 # Recommend Outfit
def recommend_outfit(uploaded_path, dataset_paths, dataset_features):
    query_features = extract_features(uploaded_path)
    sims = cosine_similarity([query_features], dataset_features)[0]
    top_indices = sims.argsort()[-5:][::-1]
    results = [(dataset_paths[i], sims[i]) for i in top_indices]
    return results

# Test using one image from your dataset
test_image = image_paths[0]
results = recommend_outfit(test_image, image_paths, features)

print("Top similar images:")
```

```
for r in results:
    print(r)
```

Show hidden output

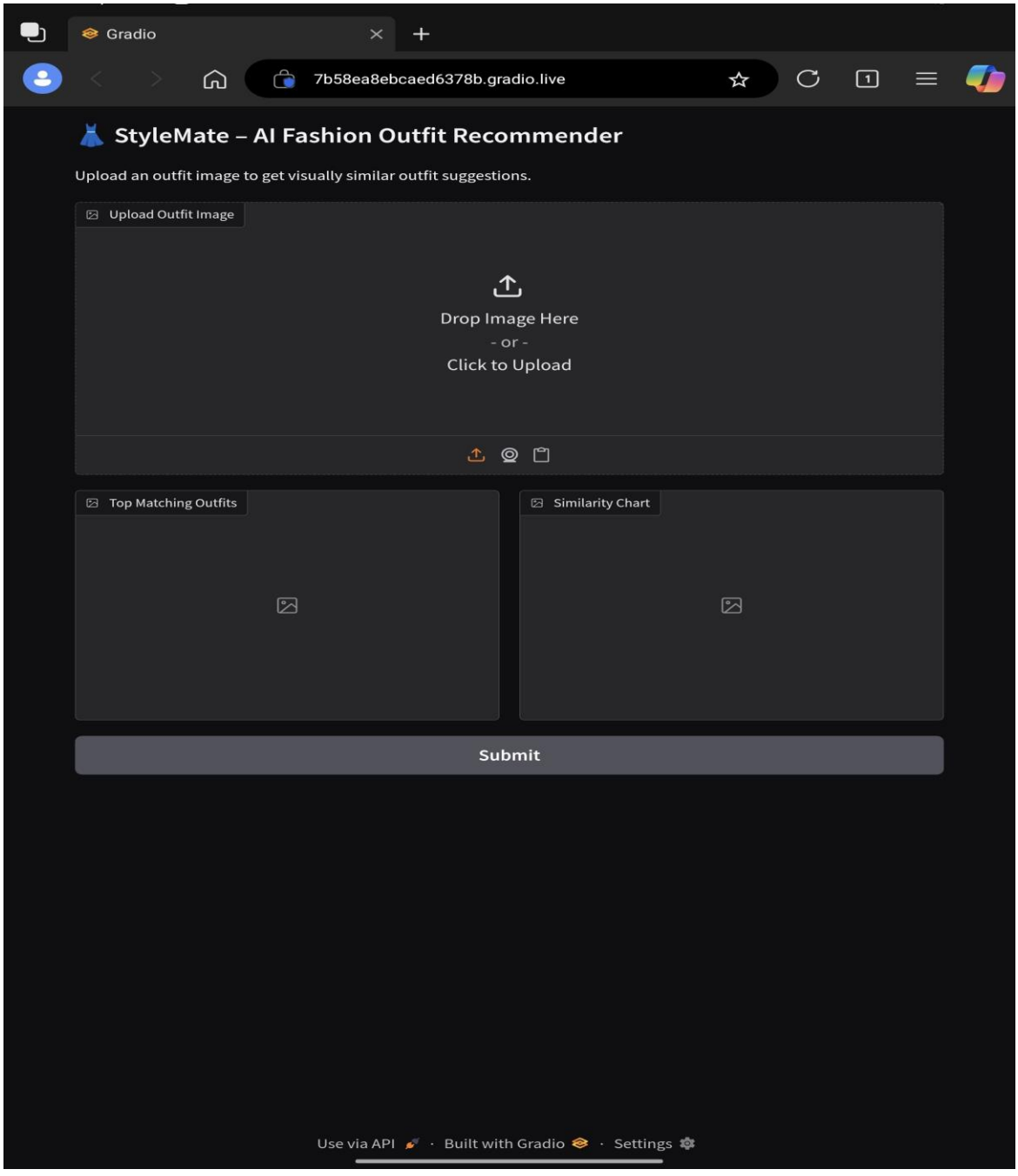
```
# Similarity Graph
import matplotlib.pyplot as plt
import gradio as gr
import numpy as np

# Function to visualize graph for similarities
def plot_similarity_chart(results):
    files = [os.path.basename(f[0]) for f in results]
    scores = [f[1] for f in results]
    plt.figure(figsize=(6, 3))
    plt.barh(files, scores, color='skyblue')
    plt.xlabel("Similarity Score")
    plt.title("Top Matching Outfits")
    plt.tight_layout()

    # Save chart to temporary file
    plt.savefig("similarity_chart.png")
    plt.close()
    return "similarity_chart.png"

# Main Gradio function
def outfit_recommender(uploaded_image):
    if uploaded_image is None:
        return [], "No image uploaded."

    # Save uploaded image
    uploaded_image.save("found_outfits/temp.jpg")

    # Run recommendation
    results = recommend_outfit("found_outfits/temp.jpg", image_paths, features)

    # Prepare output
    output_images = [r[0] for r in results]
    chart = plot_similarity_chart(results)
    return output_images, chart

# Build Gradio Interface
with gr.Blocks() as app:
    gr.Markdown("## 👗 StyleMate – AI Fashion Outfit Recommender")
    gr.Markdown("Upload an outfit image to get visually similar outfit suggestions.")

    with gr.Row():
        input_image = gr.Image(label="Upload Outfit Image", type="pil")

    with gr.Row():
        output_gallery = gr.Gallery(label="Top Matching Outfits")
        output_chart = gr.Image(label="Similarity Chart")

    submit_btn = gr.Button("Submit")
    submit_btn.click(outfit_recommender, inputs=[input_image], outputs=[output_gallery, output_chart])

app.launch(debug=True)
```

Show hidden output

Next steps: Deploy to Cloud Run

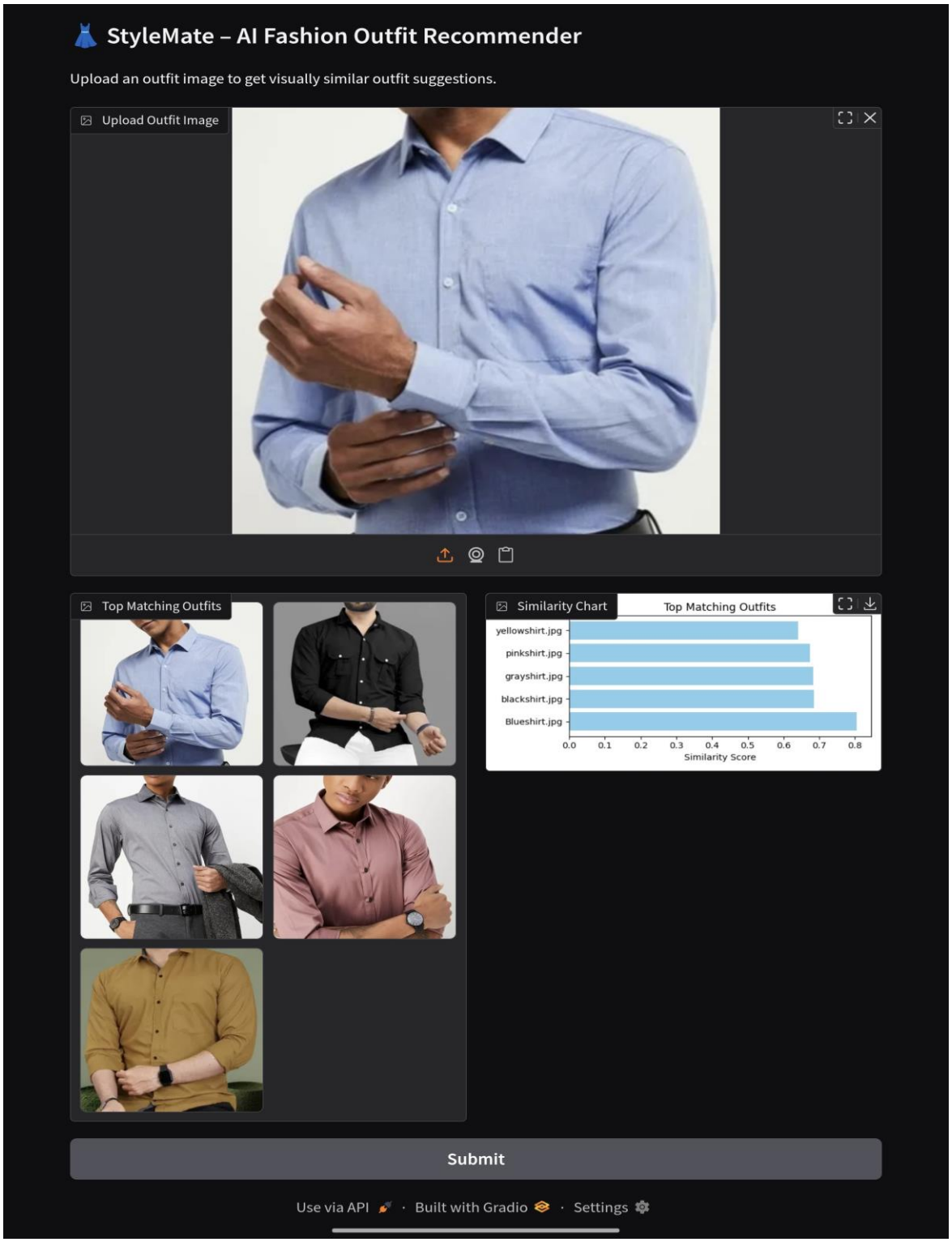
8. OUTPUT & SCREENSHOTS

8.1 Application Interface & Screenshot

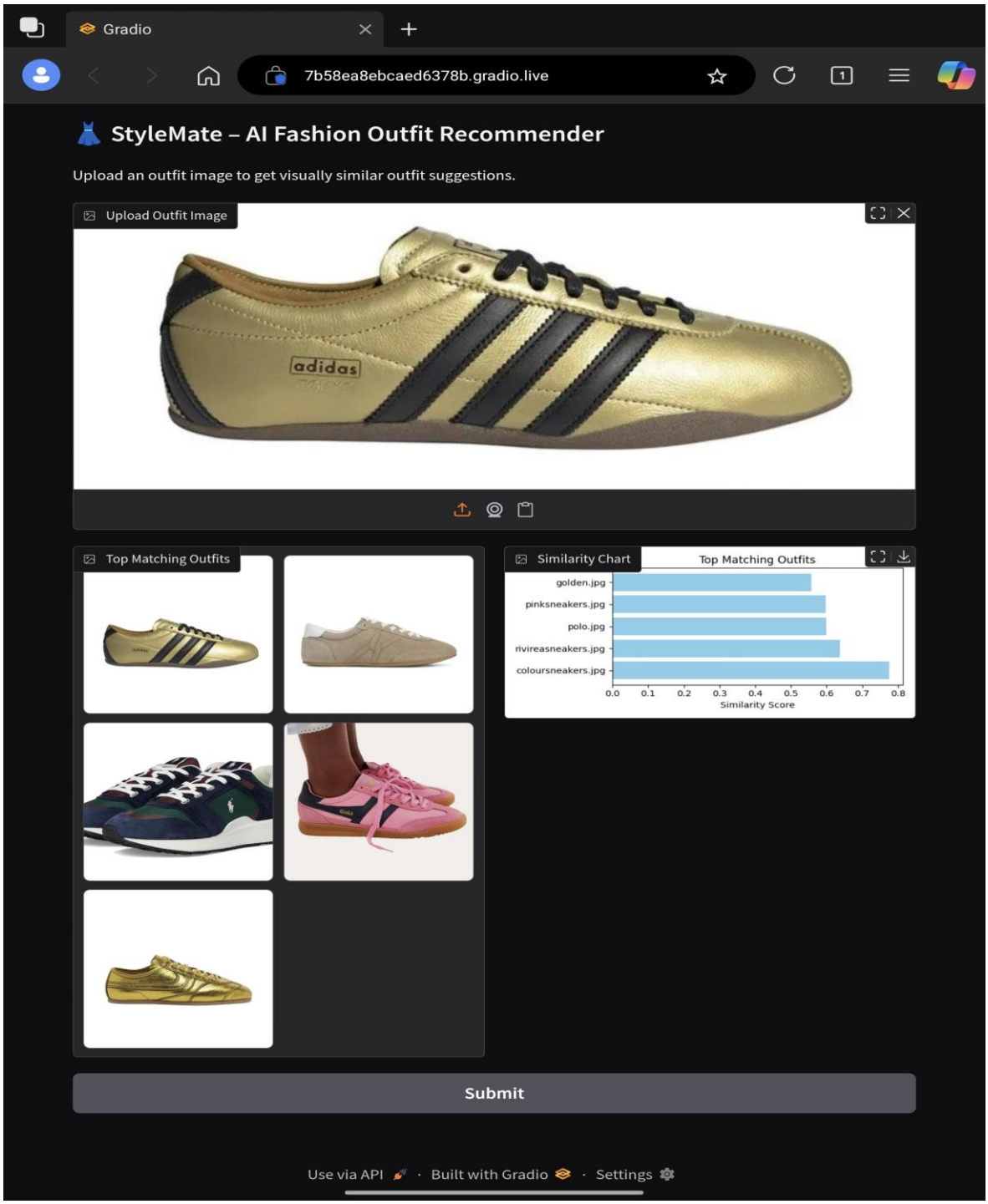


8.2 Application Output & Screenshot

Output 1:

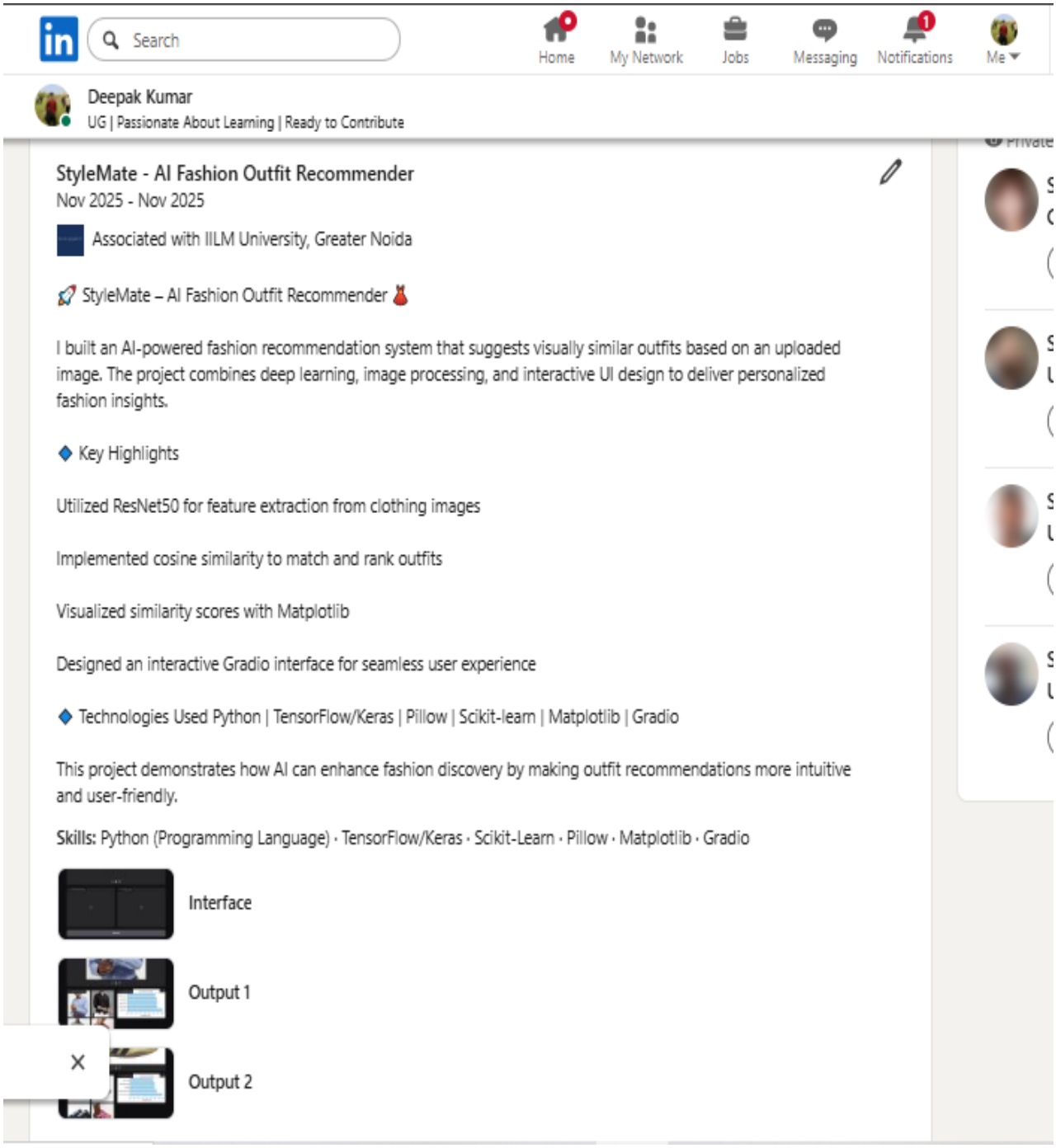


Output 2:

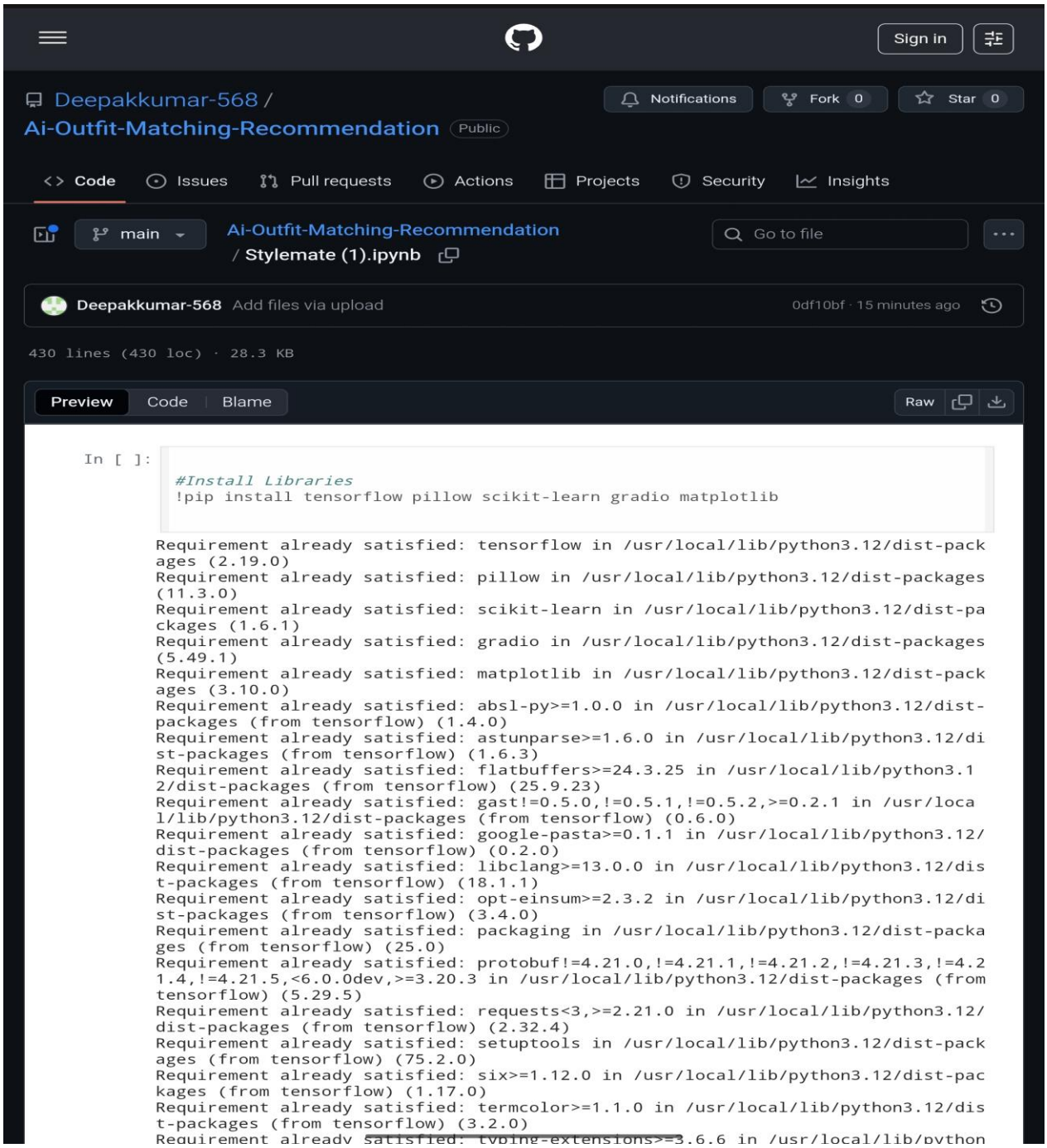

Gradio
7b58ea8ebcaed6378b.gradio.live
StyleMate – AI Fashion Outfit Recommender
Upload an outfit image to get visually similar outfit suggestions.
Upload Outfit Image
adidas
Top Matching Outfits
Similarity Chart
Top Matching Outfits
golden.jpg
pinksneakers.jpg
polo.jpg
rivireasneakers.jpg
coloursneakers.jpg
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
Similarity Score
Submit
Use via API
Built with Gradio
Settings

9. LINKEDIN & GITHUB PROOF

9.1 LinkedIn Post Screenshot



9.2 GitHub Repository Screenshot



10. CONCLUSION

The **StyleMate – AI Fashion Outfit Recommender** successfully demonstrates how deep learning and image processing can be applied to enhance fashion discovery. By leveraging a pretrained ResNet50 model for feature extraction and cosine similarity for visual matching, the system provides accurate and intuitive outfit recommendations. The integration of a user-friendly Gradio interface makes the tool accessible and interactive, allowing users to explore fashion styles with ease. This project highlights the potential of AI in personal styling and opens doors for future enhancements such as metadata filtering, real-time feedback, and deployment on web platforms.

---

11. REFERENCES

- Python Documentation
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- TensorFlow Keras Applications – Official documentation for model loading and preprocessing.
- Scikit-learn – Cosine similarity function for feature comparison.
- Gradio – Python library for building interactive ML demos.
- Matplotlib – Visualization library used for plotting similarity charts.